

Sistema de Monitorização de Equipamentos

Trabalho Prático

Autor:
José Guerreiro Faísca

1 Introdução

O objetivo deste trabalho é o desenvolvimento de um sistema de monitorização, utilizando como base o código contido no repositório web-project-1 [\[1\]](#), bem como o modelo de negócio [\[2\]](#) e análise de requisitos [\[3\]](#) aí disponíveis, que detalham a relação entre clientes, equipamentos e os seus sensores térmicos.

A solução assenta numa arquitetura cliente-servidor, onde existe uma clara separação de responsabilidades. A tarefa do servidor consiste em processar a lógica de negócio e gerir o estado dos recursos, expondo uma API REST. A persistência é assegurada pelo motor de base de dados SQLite, permitindo que o estado de todos os recursos (equipamentos e clientes) seja mantido de forma segura num ficheiro local.

Pretende-se acrescentar novas funcionalidades a desenvolver ao nível do servidor e cliente. O sistema deixará de ser um registo estático para se tornar numa plataforma dinâmica onde os equipamentos têm um ciclo de vida e uma monitorização ativa de alertas.

A persistência assegurada pelo SQLite exige agora uma atualização estrutural das tabelas para suportar a nova dinâmica.

2 Trabalho a Realizar

Clonar o repositório web-project-1 [\[1\]](#), e implementar as seguintes extensões funcionais e técnicas:

A. Gestão do Ciclo de Vida (API REST)

De acordo com requisitos de flexibilidade, a solução deve ser capaz de manipular os recursos de forma dinâmica:

- Remoção de Equipamentos: Implementar o método HTTP DELETE para permitir que um equipamento seja removido permanentemente da base de dados quando o seu ciclo de vida termina.
- Mudança de Propriedade (Cliente): Implementar o método HTTP PATCH para atualizar a associação entre um equipamento e um cliente, permitindo a transferência de recursos entre diferentes entidades do modelo de negócio. Como alternativa utilize o método PUT para substituir o recurso na íntegra.
- Interface Cliente: A componente cliente deve permitir realizar as referidas operações através de uma interface intuitiva.
- Segurança JWT: As operações de modificação (PUT, PATCH, DELETE, POST) devem exigir obrigatoriamente um Token JWT válido no cabeçalho da mensagem.
- Validação: O servidor deve validar a existência dos recursos e retornar códigos de estado apropriados (ex: 404 para não encontrado, 401 para não autorizado, 400 para quando os dados do JSON estão mal formados ou faltam campos obrigatórios).

B. Persistência e Integridade (SQLite)

A base de dados deve ser atualizada para refletir o esquema relacional do modelo de negócio:

- Relacionamentos: Garantir a integridade referencial através de chaves estrangeiras entre as tabelas de equipamentos e clientes.
- Controle de Escrita: Assegurar que cada operação de alteração (DELETE ou UPDATE) seja confirmada através do método *connection.commit()* para garantir que os dados não se perdem no encerramento do servidor.
- Base de Partida: Utilizar o script de implementação v3 [\[4\]](#) como referência para o esquema relacional base de equipamentos e clientes.

C. Sistema de Alertas

Para responder aos requisitos, o sistema deve monitorar ativamente as temperaturas:

- Monitorização Ativa: Embora o equipamento recolha os dados, o servidor atua como elemento centralizador que aplica as regras de monitorização. O servidor deve assim validar se as temperaturas reportadas estão dentro de um “intervalo razoável” para cada tipo de sensor.
- Registo de Alerta: Sempre que o servidor validar uma temperatura fora do “intervalo razoável” deve registar um alerta na base de dados.
- Pedidos HTTP assíncronos: o sistema de alertas deve processar pedidos de forma assíncrona para que a monitorização de outros equipamentos continue sem interrupções.
- Persistência dos Dados: Em caso de anomalia, o servidor deve garantir a integridade referencial e registar o alerta na base de dados, utilizando transações para assegurar que os dados são gravados permanentemente. Acrescentar novas tabelas (*limites_sensores* e *alertas_sensores*) [4] para suportar esta funcionalidade.
 - a) *limites_sensores*: Armazenamento de *min_val* e *max_val* por tipo de sensor (*temp0*, *temp1*, *temp2*), evitando introduzir valores no código.
 - b) *alertas_sensores*: Registo histórico das anomalias, garantindo que o alerta enviado possa ser consultado via API REST.
- Consulta Ativa (pull): O sistema deve garantir um mecanismo de consulta ativa de alertas:
 - a) Ao nível do Servidor (Backend)
 - Rota de Alertas: Criar a rota GET /api/alertas que devolve o histórico de anomalias em formato JSON.
 - Lógica de Dados: A função associada à rota deve consultar a tabela *alertas_sensores* e retornar os dados no formato JSON
 - Segurança: Esta rota deve ser protegida, exigindo que o cliente envie um token JWT válido no cabeçalho.
 - b) Ao Nível do Cliente (Frontend - Interface)
 - Modelo de Interrogação (polling): O sistema deve permitir a consulta manual de alertas sem necessidade de recarregar a página.
 - Interface Cliente: A página web deve conter um elemento de interface (ex: um botão "Atualizar Alertas") e uma zona dedicada (ex: uma ou <div>) para apresentar a lista.
 - Lógica JavaScript Assíncrona: Deve ser utilizada a Fetch API com a sintaxe *async/await* para consultar periodicamente (ou via interação do utilizador) o estado dos alertas.
 - Tratamento de exceções: Implementar blocos *try...catch* para gerir erros de rede ou falhas na API.
 - Manipulação do DOM: Após receber o JSON, a função deve percorrer os alertas e atualizar dinamicamente o HTML da página, injetando novo elementos sem interromper a experiência do utilizador.

D. Intercâmbio de Informação (JSON e GeoJSON)

- JSON: Toda a comunicação entre cliente e servidor, tanto na API REST como no sistema de alertas, deve utilizar obrigatoriamente o formato JSON, garantindo: a) o corpo das mensagens é serializado no formato JSON; b) o campo *Content-Type* do cabeçalho HTTP é definido como *application/json* em todas as mensagens.
- GeoJSON (opcional): Para visualização geográfica da localização dos equipamentos, pode ser integrado o suporte a GeoJSON.

E. Registo da Leitura de Sensores

- Ambiente de Desenvolvimento: O registo da temperatura é realizado via script *temp_rand_main.py* [5] que simula os sensores, gerando marca temporal e valores aleatórios de temperatura e enviando-os ao servidor através da rota pré-definida */temperatura*. No script, ajustar variáveis *host* e *port*, se necessário. Executar passando como argumento o identificador do equipamento, previamente registado na base de dados. Este script deve ser reprogramado de modo a usar o Token JWT para enviar dados.
- Ambiente de Produção: O registo da temperatura é efetuado por Microcontroladores e Sensores reais, através da rede local (LAN) ou da Internet (WAN), utilizando a mesma rota */temperatura*, disponibilizada no ambiente de desenvolvimento.

F. Ferramentas de Teste e CLI

- Documentação: Integrar a especificação Swagger/OpenAPI para teste das rotas via */api/docs*.
- Desenvolvimento: Programar um script independente em Python ou Node.js que simule a criação de equipamentos e a verificação de alertas via terminal, lendo um ficheiro JSON local com dados, enviando-os para a API do servidor, garantindo a automação dos testes do sistema. Este script deve usar o Token JWT para enviar dados.

G. Ambientes e Tecnologias do Cliente

O trabalho deve contemplar a implementação do cliente adaptada a dois cenários distintos:

- Ambiente Móvel: O cliente deve ser acessível via Browser (otimizado para dispositivos móveis).
- Ambiente PC/Desktop: O cliente pode basear-se numa interface web via Browser ou numa aplicação dedicada, desenvolvida para ambiente gráfico (GUI), ou linha de comandos.

H. Persistência e Segurança

- Integridade Referencial: Garantir que o cliente respeite as restrições definidas na base de dados SQLite, em particular as chaves estrangeiras entre equipamentos e clientes, evitando operações que violem essas relações.
- Validação de Input: O cliente deve realizar validações básicas dos dados antes de os enviar para a API, prevenindo o envio de valores malformados ou incompletos.

Entrega

Este trabalho poderá ser produzido individualmente ou em grupo. A entrega deverá ser realizada via Moodle, através de um ficheiro zip contendo todos os ficheiros relativos ao trabalho, e um relatório (obrigatório) no formato PDF que descreva claramente a lista de ficheiros entregues e resuma os testes realizados, obedecendo OBRIGATORIAMENTE ao seguinte formato: **<nr_aluno(s)>-trabalho-pratico.zip**

Referências

- [1] [Repositorio web-project-1](#)
- [2] [Modelo de Negócio](#)
- [3] [Análise de Requisitos](#)
- [4] [Script de Implementação v3](#)
- [5] [Script de registo de temperatura](#)